

CS 577 Deep Learning

Homework 2: Inside an MLP—Forward, Backward, and Gradient Evidence

Assignment Specification and Grading Rubric

Fall 2026

Release:	Monday, August 31, 2026
Due:	Sunday, September 27, 2026, 11:59 p.m. Central Time
Weight:	8% of the course grade
Expected effort:	8–10 focused hours
Individual work:	This homework must be completed and submitted individually.

Learning Goals

After completing this homework, you should be able to:

1. trace values and shapes in a batched two-layer MLP forward pass;
2. derive binary-cross-entropy gradients and backpropagate through affine and ReLU layers;
3. implement a vectorized NumPy MLP forward and backward pass without autograd;
4. validate manual gradients with centered finite differences and PyTorch autograd; and
5. explain experimentally why a nonlinear hidden layer can solve a problem that a linear classifier cannot.

The goal is not to memorize four gradient formulas. The equation, implementation, shape trace, and numerical evidence should tell the same story.

Files to Submit

Submit one ZIP file named `hw2_<hawk_username>.zip` containing exactly:

1. `hw2_<hawk_username>.pdf` — compiled from the written template;
2. `hw2_<hawk_username>.ipynb` — the completed notebook with all outputs visible; and
3. `hw2_mlp.py` — your completed NumPy implementation.

Run `python -m unittest -v test_hw2_public.py` from the HW2 directory before submission. Passing public tests is useful evidence but does not guarantee full credit. Hidden tests use additional inputs, and grading also considers derivations, explanations, plots, and code quality.

Before submitting, restart the notebook kernel and run all cells from top to bottom. It must finish without an uncaught exception on the CPU path.

Collaboration, References, and Generative AI

You may discuss course concepts and general debugging strategies, but you must independently derive, implement, test, and submit your work. Do not share code, completed equations, numerical answers, or solution-producing prompts.

Course materials and official NumPy, PyTorch, Python, Matplotlib, and LaTeX documentation are allowed. Generative-AI tools may be used for conceptual explanations, syntax reminders, documentation search, or debugging suggestions. They may not produce the core derivations, MLP implementation, gradient analysis, experiment interpretation, or written answers submitted for credit.

Every submission must include an assistance/AI disclosure. A valid disclosure names the tool or person, the purpose, and what you verified or changed. If none was used, write: **No external assistance or generative-AI tools were used.**

Model and Notation Used Throughout

For B examples, D input features, and H hidden units:

$$X \in \mathbb{R}^{B \times D}, \quad W_1 \in \mathbb{R}^{D \times H}, \quad b_1 \in \mathbb{R}^H, \quad W_2 \in \mathbb{R}^{H \times 1}, \quad b_2 \in \mathbb{R}, \quad y \in \{0, 1\}^{B \times 1}.$$

The network is

$$Z_1 = XW_1 + b_1, \quad A_1 = \text{ReLU}(Z_1), \quad S = A_1W_2 + b_2.$$

S contains logits, not probabilities. The probability of class 1 is $\sigma(S)$. Use mean binary cross-entropy with logits:

$$J = \frac{1}{B} \sum_{i=1}^B \left[\max(s_i, 0) - y_i s_i + \log \left(1 + e^{-|s_i|} \right) \right].$$

Unless stated otherwise, use NumPy `float64`. Do not add regularization, dropout, momentum, or Adam to the required implementation.

Part 0: Conventions and Reproducible Setup (5 points)

0.1 Environment and Public-Test Report (2 points)

In the notebook, report Python, NumPy, and PyTorch versions and show the public-test output. Use CPU for required comparisons.

0.2 Shape and Reduction Contract (3 points)

State why targets are stored as $(B, 1)$ rather than $(B,)$. Explain what incorrect shape can result from combining $(B,)$ and $(B, 1)$. State where the factor $1/B$ enters backward propagation when loss is a batch mean.

Part 1: Forward and Backward Mathematics (20 points)

Complete the provided LaTeX written report.

1.1 Forward Shape Trace (6 points)

For $X, W_1, b_1, Z_1, A_1, W_2, b_2, S, y$, and J , state the shape and explain both bias broadcasts. Distinguish logits from probabilities.

1.2 From BCE-with-Logits to the First Backward Signal (6 points)

For one example, start from

$$\ell(s, y) = -y \log \sigma(s) - (1 - y) \log(1 - \sigma(s))$$

and derive

$$\frac{\partial J}{\partial S} = \frac{\sigma(S) - y}{B}.$$

Show enough intermediate reasoning to make the cancellation clear. Explain why the derivative remains the same when the forward loss uses an algebraically equivalent stable expression.

1.3 Backpropagate Through the Complete MLP (8 points)

Starting from $dS = \partial J / \partial S$, derive the output-layer gradients dW_2 and db_2 , the hidden signals dA_1 and dZ_1 , and the input-layer gradients dW_1 and db_1 . Optionally derive dX . Give every tensor shape. Explain why bias gradients reduce over the batch axis and retain their parameter shapes.

Part 2: NumPy Forward and Backward Implementation (30 points)

Implement the required functions in `hw2_mlp.py`. Do not use PyTorch, JAX, TensorFlow, autograd packages, symbolic differentiation, or finite differences inside `mlp_loss_and_gradients`.

2.1 Stable Scalar/Vector Primitives (6 points)

Implement:

- `sigmoid_stable(z)` without overflow for large $|z|$;
- `relu(z)`; and
- `binary_cross_entropy_with_logits(logits, y)` using the stable expression above.

The loss must validate matching $(B, 1)$ shapes and return one Python `float`.

2.2 Vectorized Forward Pass (10 points)

Implement `mlp_forward(X, params)` and return `(logits, cache)`. Cache the intermediates needed by backward propagation. Do not loop over examples or hidden units, and do not mutate inputs or parameters. Demonstrate the supplied fixed check case and report shapes and logits.

2.3 Manual Backward Pass (14 points)

Implement `mlp_loss_and_gradients(X, y, params)`. It must run forward, compute mean BCE-with-logits, manually backpropagate through output affine, ReLU, and input affine operations, and return `(loss, grads)`. The gradient dictionary must contain exactly `W1`, `b1`, `W2`, and `b2`, with shapes matching the parameters. Do not update parameters inside this function. Report fixed-case loss, gradient shapes, and gradient norms.

Part 3: Two Independent Gradient Checks (20 points)

Use the fixed notebook case. Its hidden preactivations are separated from zero so ReLU's derivative is unambiguous.

3.1 Centered Finite Differences (10 points)

Implement `finite_difference_gradients(X, y, params, epsilon=1e-5)`. Perturb one scalar parameter at a time and return a dictionary matching `params`. Leave the input dictionary unchanged.

For each parameter report maximum absolute difference and relative error:

$$\frac{\|g_{\text{manual}} - g_{\text{reference}}\|_2}{\|g_{\text{manual}}\|_2 + \|g_{\text{reference}}\|_2 + 10^{-12}}.$$

3.2 PyTorch Autograd Comparison (10 points)

Recreate the same MLP and parameters using PyTorch `float64`, compute the same mean BCE-with-logits, call `backward()`, and compare PyTorch gradients with manual NumPy gradients. PyTorch is an independent reference; do not copy its gradients into your NumPy implementation. Report a comparison table and explain what a passing check establishes—and what it does not establish.

Part 4: Nonlinear Experiment: When a Hidden Layer Matters (20 points)

The notebook supplies a deterministic noisy XOR-style dataset. Use the same train/test split in every comparison.

4.1 Establish a Linear Baseline (6 points)

Train the provided logistic-regression baseline. Report train/test loss and accuracy. Plot its decision boundary and explain why one affine score followed by sigmoid cannot separate alternating XOR regions with one straight boundary.

4.2 Train the Manual MLP (8 points)

Use your NumPy backward pass in `train_mlp`. Unless a justified change is documented, use hidden width $H = 8$, full-batch gradient descent, 2,000 steps, learning rate 0.2, and seed 577. Plot training loss and the learned decision boundary. Report train/test loss and accuracy.

4.3 Interpret the Evidence (6 points)

In 5–8 sentences address: what features hidden ReLU units create; why the output layer is linear in hidden features; whether the evidence indicates learning, underfitting, or overfitting; and why one run does not guarantee success for every initialization or hyperparameter choice.

Part 5: Reflection and Submission Quality (5 points)

Answer each in 2–4 sentences:

1. Which cached value was most useful while implementing backward propagation, and why?
2. If a gradient check fails, what three possibilities would you investigate first?
3. Why should manual, finite-difference, and autograd gradients be close but not necessarily bit-for-bit identical?

The final points also include readable code, meaningful names, visible outputs, a clean PDF, correct filenames, and the required disclosure.

Grading Rubric

Part	Points	Full-credit evidence
0. Conventions/setup	5	Environment/test report; correct $(B, 1)$ and mean-reduction explanation
1. Mathematics	20	Shape trace; BCE derivative; complete vectorized backward chain with shapes
2. NumPy implementation	30	Stable primitives; vectorized forward; correct manual gradients; no prohibited shortcut
3. Gradient verification	20	Centered finite differences; faithful PyTorch recreation; per-parameter error evidence
4. Nonlinear experiment	20	Fair baseline; successful manual-MLP training; plots; evidence-based analysis
5. Reflection/quality	5	Specific reflection; readable code; visible outputs; filenames; disclosure
Total	100	

Common Deductions

- Notebook does not run from a clean restart: up to -15 , depending on severity.
- Manual backward uses autograd, finite differences, or another differentiation package: no credit for affected manual-backward items.
- Missing PDF, notebook, or Python file: up to the value of the affected component.
- Hidden/missing outputs: -1 to -5 .
- Loops over examples/hidden units where vectorization is required: up to half credit for the affected function.
- Correct gradient values with incorrect parameter shapes: up to half credit for that gradient.
- Missing reasoning where requested: at most half credit for the subpart.
- Missing assistance/AI disclosure: -2 and submission returned for correction if necessary.

Reference Numerical Tolerances

- Stable primitives/forward fixed cases: `atol=1e-10`, `rtol=1e-9`.
- Manual gradients: `atol=1e-9`, `rtol=1e-8` away from ReLU's kink.
- Centered finite differences ($\epsilon = 10^{-5}$): maximum absolute and relative error below approximately 2×10^{-6} per parameter on the supplied case.
- Manual versus PyTorch `float64`: maximum absolute error below approximately 10^{-9} .
- Required XOR experiment: the MLP should substantially outperform the linear baseline and normally reach at least 90% test accuracy. The threshold is interpreted with implementation and plots, not used as the entire grade.